Homework Assignment- Risk-Adaptive Access Control with Synthetic Enterprise Logs

This assignment asks students to design an AI-assisted access-control engine for a realistic enterprise setting. Rather than using a fixed dataset, students must generate or curate their own logs, build a static policy baseline, train a risk-scoring model, convert risk into allow, challenge, or block decisions, and critically examine the governance issues created by automated access control.

Learning Objectives

- Model users, resources, context, and risk signals in an access-control setting.
- Generate and validate synthetic behavioural data with both normal and anomalous activity.
- Compare static rules with an adaptive AI-assisted decision engine.
- Explain model decisions in a way that supports security review and auditability.
- Assess policy drift, lockout risk, and governance concerns in automated security systems.

Provided Material

- No mandatory instructor material is provided.
- Students must create their own data generator or data-assembly pipeline and must document every schema field used in the project.
- Open-source libraries, publicly documented schemas, or public security datasets may be used as inspiration or as a supplement, but the final submission must remain self-contained and reproducible.

Operational / Ethical Constraints

- This exercise must use synthetic or appropriately anonymised data only.
- Automated access decisions should be treated as experimental outputs, not as live policy changes on production systems.

Required Tasks

1. Design a Synthetic Enterprise Scenario

Create a realistic organisational setting so that access decisions have meaningful context.

Steps:

- Define an organisation with at least 50 users spread across at least four roles or departments.
- Define at least 25 resources or services with sensitivity labels, such as public, internal, confidential, or privileged.
- Generate at least 14 days of access history with a minimum of 12,000 events overall unless a smaller but well-justified design is used.
- The log schema must include, at minimum: timestamp, user identifier, role, department, location or network zone, device identifier, resource identifier, action, outcome, and a ground-truth label indicating normal or anomalous behaviour.

- Document all assumptions that shape the organisation, the schedule patterns, and the normal behaviour of each role.

Minimum evidence expected:

- A data-generation script or clearly reproducible data-construction process.
- A schema description that another student could use without guessing field meaning.

2. Inject Diverse Security-Relevant Anomalies

Create anomalies that are realistic enough to test both policy logic and behavioural modelling.

Steps:

- Inject at least five anomaly categories.
- Recommended categories include impossible travel, off-hours privileged access, first-seen device for a sensitive action, unusual resource access for the role, bulk file access or data volume spikes, or suspicious privilege escalation behaviour.
- Ensure that anomalies are not all reducible to one obvious field; at least some anomalies should depend on context or historical behaviour.
- Keep the anomaly rate realistic; as a guide, between 3 percent and 10 percent of the total events is usually appropriate unless otherwise justified.

Minimum evidence expected:

- A short anomaly-design note explaining how each anomaly was constructed and what security question it tests.

3. Implement a Static Baseline Policy Engine

Build a non-AI baseline so that the value of adaptive methods can be measured rather than assumed.

Steps:

- Implement a simple role-based or rule-based access-control engine using the same input logs.
- The baseline must produce at least allow and block decisions, and may optionally include challenge or review states.
- Document the rules in a concise and auditable form.
- Evaluate how the baseline handles both normal events and the injected anomalies.

Minimum evidence expected:

- A baseline decision file or output log.
- A short justification of why the baseline rules are reasonable but limited.

4. Build an AI Risk-Scoring Model

Create a model that assigns a risk score to each access request or event.

Steps:

- Engineer behavioural and contextual features, such as peer-group rarity, device novelty, time-of-day deviation, recent failure count, access frequency, sensitivity mismatch, and session volume patterns.
- Train a model that outputs a score or probability rather than only a hard label when possible.
- Explain whether the problem is approached as supervised classification, anomaly detection, or a hybrid design.
- State how temporal leakage is prevented and how the train-test split respects time or user boundaries where relevant.
- Normalise or calibrate the final score to a clear range, such as 0 to 100.

Minimum evidence expected:

- A reproducible training pipeline and a clear description of the final feature set.
- A risk score for every event in the evaluation set.

5. Convert Risk into Dynamic Access Decisions

Map the model output into a security-relevant decision process rather than stopping at classification.

Steps:
- Define at least three outcomes: allow, challenge, and block.
- Implement a decision policy that maps the risk score and contextual factors into those outcomes.
- Include at least one step-up control such as MFA challenge, manager approval, or delayed review for medium-risk cases.
- Produce an output file containing at minimum: event identifier, risk score, final decision, and a short explanation or reason code.

Minimum evidence expected:
- A decisions output file that is easy for a marker to inspect.
- A short explanation of how thresholds were chosen.

6. Evaluate Quality, Explainability, and Operational Risk

Assess both predictive performance and whether the resulting system could be used responsibly.

Steps:
- Report precision, recall, F1-score, and either ROC-AUC or PR-AUC as appropriate for your class balance.
- Compare the AI-assisted system against the static baseline using the same evaluation set.
- Provide at least five event-level explanations showing why the model judged the case to be high or medium risk.
- Analyse the cost of false positives and false negatives in operational terms, for example accidental lockouts versus missed insider activity.
- Break down results by role, department, or resource sensitivity and comment on whether performance differs significantly across groups.

Minimum evidence expected:
- A comparative evaluation section with both quantitative and qualitative evidence.

7. Simulate Drift and Governance Challenges

Show that the system can be stress-tested beyond one static dataset snapshot.

Steps:
- Introduce at least one drift scenario, such as remote-work week, temporary project access, a department reorganisation, or a new-device rollout.
- Re-evaluate the system before and after the drift scenario.
- Discuss whether the issue is best handled by retraining, threshold recalibration, rule updates, or governance review.
- Reflect on explainability, auditability, and the risk of policy drift if the system changes decisions automatically over time.

Minimum evidence expected:
- A concise discussion of how the system behaves under change rather than only under the original distribution.

Deliverables

- The data generator or data assembly pipeline.
- The baseline policy engine and the AI risk-scoring pipeline.
- A decisions output file and, if relevant, a scores file for the evaluation set.
- A structured report covering scenario design, anomaly design, baseline policy, model design, evaluation, explainability, drift analysis, and limitations.
- Screenshots, plots, or tables that make the results easy to inspect.
- A README and requirements file or equivalent setup instructions.

Optional Advanced Tasks

- Perform a fairness analysis across departments, user groups, or resource classes and discuss whether the decision system disadvantages one group disproportionately.
- Implement online or incremental adaptation and compare it with batch retraining under the same drift scenario.
- Represent users and resources as a graph and test whether graph-based features improve detection of unusual access paths.
- Build a lightweight dashboard that lets an analyst inspect high-risk events, explanations, and proposed policy changes.